School of Informatics

Informatics Project Proposal

Controlled Component-Level Evaluation of AI Agents via Modular Benchmarking

UID: B283223

Date: April 2026

Tutor: James Garforth

Supervisor: Luo Mai

I give permission for future Informatics Project Proposal classes to read an anonymous version of this document.

This page does not count towards the page limit.

Abstract

This project investigates how memory, retrieval, and scheduling components shape AI-agent performance under controlled conditions. It proposes a modular benchmarking framework with containerised mock MCP servers, fixed tasks, a fixed model backbone, and repeatable seeded runs. A factorial experimental design compares component variants using both task outcomes and trajectory-level diagnostics such as progress, repetition, goal drift, and recovery behaviour. The expected outcome is reproducible evidence, within a controlled benchmark setting, about how architectural choices affect reliability and failure behaviour, together with a reusable evaluation testbed for future component-level agent studies.

Declaration of Own Work

I hereby declare that the work presented in this document is my own. All sources used in preparing the proposal are acknowledged in the text and reference list. Any use of generative AI tools has been limited to drafting support, language editing, and literature organisation; all substantive research decisions, analysis, and final written content remain my responsibility.

This page does not count towards the page limit.

1. Motivation

Large language model (LLM) systems are increasingly deployed as agents that coordinate planning, memory, retrieval, and external tools over multi-step tasks.[1] Evaluating such systems differs from evaluating static language understanding because performance depends on sequential decisions, environment feedback, and failure recovery over extended trajectories.[1] In practical settings, weak evaluation can overestimate reliability and allow errors to propagate through long-horizon workflows.[1] Benchmarking research therefore increasingly asks not only what a model knows, but what it can reliably do under dynamic conditions.[1]

Since 2023, benchmark design has expanded from narrow task evaluation to heterogeneous interactive ecosystems.[1] AgentBench evaluates LLMs as agents across diverse interactive environments.[2] OpenHands positions generalist software-development agents as an open evaluation platform.[3] BrowserGym standardises web-agent evaluation in a unified environment.[4] WorkArena targets realistic knowledge-work tasks in enterprise-style interfaces.[5] Gorilla focuses on accurate API-call use and reduced hallucination in tool invocation.[6] StableToolBench is designed for stable large-scale benchmarking of tool learning.[7] ToolSandbox adds stateful conversational tool-use evaluation.[8] MCP-Universe extends benchmarking to realistic long-horizon tasks over MCP servers.[9]

Methodological issues nevertheless persist in agent benchmarking.[10] First, many studies remain end-to-end, which makes it difficult to attribute observed performance differences to specific internal components.[11] Second, realism often conflicts with reproducibility because unstable online APIs and changing API status can distort cross-run comparisons.[7]

This project addresses these gaps through controlled, component-level evaluation.[11] It holds the model backbone, task definitions, and environment fixed while systematically varying memory, retrieval, and scheduling strategies. By combining outcome measures with trajectory diagnostics such as progress, repetition, goal drift, and recovery behaviour, the study aims to produce causally interpretable and reproducible evidence for component attribution under controlled benchmark conditions.[11] This focus is timely because recent benchmark guidance emphasises scientifically rigorous evaluation for agentic systems.[10]

Research questions. The project is organised around two explicit research questions:

1. **RQ1:** under a fixed model backbone, task suite, and tool interface, what main and interaction effects do memory, retrieval, and scheduling components have on task success, cost, and failure behaviour?
2. **RQ2:** do trajectory-level metrics provide more informative and actionable component-level diagnosis than outcome-only metrics in this controlled setting?

2. State of the Art

Research on agent evaluation has expanded rapidly since 2023.[1] The literature nevertheless remains fragmented across benchmark families and evaluation philosophies.[1] This proposal therefore compares prior work along three axes: environmental realism versus experimental control, end-to-end capability testing versus component observability, and outcome-only scoring versus process-level diagnostics. This lens is useful because recent benchmark guidance stresses explicit design trade-offs and rigorous reporting.[10]

AgentBench evaluates agents across diverse interactive environments.[2] OpenHands frames generalist software-development agents as an open platform.[3] These benchmarks offer ecological breadth, but aggregate end performance is less useful for causal attribution across memory, retrieval, scheduling, and prompt effects.[1]

Tool-use benchmarks offer finer-grained stress tests. Gorilla foregrounds tool-call correctness and hallucination reduction.[6] StableToolBench focuses on stable large-scale benchmarking of tool-learning systems.[7] ToolSandbox captures stateful conversational tool use over arbitrary trajectories.[8] MCP-Universe studies long-horizon tasks through interaction with realistic MCP servers.[9] MCP-Bench evaluates tool-using agents on complex multi-step tasks via MCP servers.[12] These platforms improve operational realism, but reproducible cross-benchmark comparison remains difficult.[10]

BrowserGym provides a unified environment for standardised web-agent evaluation.[4] WorkArena measures performance on common knowledge-work tasks.[5] WebChoreArena extends evaluation to realistic tedious web tasks.[13] MedAgentBench provides a realistic virtual EHR environment for medical agents.[14] Greater ecological realism can nonetheless amplify interface drift and reduce reproducibility over time.[7]

Metric design has moved beyond simple success rate.[1] AgentQuest shows that progress and repetition indicators reveal failure dynamics hidden by endpoint scores.[11] Safety-oriented evaluation has also become more explicit in web-agent benchmarking.[15] The field still lacks a unified metric stack that links outcome quality, process quality, and recovery behaviour.[1] Automatic-evaluator work suggests that trajectory-aware assessment can be more informative than a single pass/fail judgement.[16]

Four gaps follow directly from this literature review. Component attribution remains underdeveloped in end-to-end benchmarks.[1] The realism–reproducibility trade-off remains unresolved in live or evolving environments.[7] Diagnostic incompleteness persists where success rates dominate over trajectory and recovery metrics.[11] Many studies also under-report interaction effects and uncertainty, limiting reproducible comparison across architectures.[10]

These gaps motivate the design proposed in Section 3. Controlled modular ablations improve causal attribution.[11] Seeded experimentation, effect sizes, and explicit uncertainty reporting improve methodological rigor.[10] In this sense, the proposal does not simply apply existing benchmarks; it integrates their strongest design ideas while addressing their shared methodological weaknesses.

3. Implementation

3.1. Methodology or Approach

This project adopts a controlled intervention-based experimental design to estimate how three architectural components—memory, retrieval, and scheduling—affect agent behaviour.[11] The primary objective is component attribution rather than raw performance optimisation, so non-target variables are fixed to preserve internal validity and reduce confounding.[10]

Specifically, the following elements are held constant across all runs:

1. the LLM backbone,
2. prompt templates and system instructions,
3. task suite and input data,
4. tool interfaces and schemas, and
5. runtime configuration, including decoding parameters and the shared evaluation seed set.

These constraints are necessary because varying these elements simultaneously would introduce confounds that cannot be disentangled from component-level effects.[10]

Randomness protocol. All configurations are evaluated on the same fixed seed set and the same pre-generated perturbation schedules. Decoding settings remain fixed across runs, and the

mock environment is deterministic conditional on the selected seed and perturbation schedule. This protocol preserves matched comparisons across configurations while still allowing variance estimation from repeated trials. The resulting task–seed–schedule cells are treated as matched blocks in the downstream analysis so that pairing is preserved when estimating uncertainty and component effects.

Experimental factors and isolation. Three factors are varied, each with two levels:

- **Memory backend (Factor A).**
  - **A1:** SQLite-backed episodic memory storage.
  - **A2:** vector-index-backed episodic memory storage.

  In both A1 and A2, episodic-memory reads use the same structured query schema, metadata filters, and recency tie-breaking; semantic nearest-neighbour recall from internal memory is intentionally out of scope for this factor. Both backends therefore store the same memory records and expose the same read/write contract, so the factor isolates the storage/indexing substrate rather than the retrieval policy.
- **Retrieval strategy (Factor B).**
  - **B1:** BM25 retrieval over the external task-document corpus.
  - **B2:** embedding-based retrieval over the same task-document corpus.

  The document corpus, chunk boundaries, indexing cadence, and top-k are fixed to isolate matching strategy effects from data availability or retrieval depth.
- **Scheduling policy (Factor C).**
  - **C1:** sequential execution (fixed action order).
  - **C2:** rule-based prioritisation (heuristic reordering using tool state).

  The planner output space and action set are identical across conditions; only ordering is changed, preventing planner-policy confounding.

These controls separate internal memory storage from external document retrieval, which supports interpretable main- and interaction-effect estimation.[10]

Task suite design. Tasks are intentionally constructed to expose component-specific behaviour rather than sampled arbitrarily:

- **Memory-dependent tasks** (e.g., multi-step recall, state persistence), designed to stress long-context retention and memory access accuracy.
- **Retrieval-dominant tasks** (e.g., document lookup, fact grounding), designed to differentiate keyword and semantic matching behaviour.
- **Recovery-intensive tasks** (e.g., tool failures, partial observability), designed to evaluate scheduling and recovery strategies under disruption.

The final task suite is balanced across these families so that pooled estimates are not dominated by a single task type. Each task has a predefined success criterion and explicit state-transition structure so progress is measurable and comparable across conditions.[11] Results are reported both overall and stratified by task family so that component effects can be distinguished from task-composition effects.

Experimental design. The study uses the full 2^3 factorial over the three factors, yielding eight configurations.[10] This preserves estimability of all main and two-way interaction effects without aliasing. Repeated matched trials are used to estimate variance and improve statistical stability.[10] If compute pressure increases, the contingency plan is to reduce the number of tasks or repeated trials rather than fractionating the factor space. Main and interaction effects are estimated with response-appropriate models, with effect sizes and confidence intervals reported across runs. The primary analysis uses blocked or mixed-effects formulations that account for

task instance and matched seed–perturbation cells, and reports both pooled estimates and task-family-specific contrasts.

Trajectory-level metrics (operationalised). In addition to outcome metrics (task success, latency, token/cost usage), the study computes:

- **Goal drift:** proportion of steps that do not reduce estimated distance to completion under a predefined task-state progression function.
- **Repetition rate:** fraction of actions that repeat a previous action without state change, or after an unresolved failed attempt.
- **Recovery steps:** number of steps required to return to forward progress after the first observable failure event.

These metrics are computed from full execution traces and validated on a manually inspected subset so that they correspond to meaningful failure modes rather than logging artefacts.[11]

Diagnostic audit protocol. To evaluate `RQ2`, a sampled subset of traces is reviewed with a predefined rubric covering failure-label alignment, specificity, and actionability of the diagnostic signal. Configuration identity is masked where feasible, and a held-out subset is rescored in a second masked pass; disagreements are adjudicated against predefined task-level failure labels rather than informal preference.

Environment design and justification. A modular, containerised environment with mock MCP servers emulates databases, file systems, and APIs. This environment is chosen over live web/API settings because latency volatility, interface drift, and non-deterministic failures would reduce cross-run comparability and weaken attribution.[7]

To mitigate reduced realism, controlled perturbations are injected:

- stochastic tool delays,
- temporary tool unavailability, and
- noisy or incomplete observations.

Perturbations are sampled from fixed distributions, attached to normalised task-state or tool-event identifiers (e.g., the nth eligible call for a given tool class), and replayed consistently across configurations so that robustness and recovery can be evaluated without sacrificing comparability. If a scheduled perturbation becomes inapplicable because the corresponding event is never reached, it is logged as unused rather than reassigned to a different event.[16]

Comparison to alternative approaches.

- **Full factorial vs. reduced trial budget:** the study keeps the full 2^3 design so main and two-way interactions remain estimable; compute pressure is handled by reducing task volume or repeated trials instead of fractionating the factor space.[10]
- **Live benchmarking vs. simulated environment:** live settings improve realism but reduce reproducibility; simulation is prioritised because the research goal is attribution.[7]
- **Planner-rich orchestration vs. rule-based scheduling:** richer orchestration can improve expressiveness, but it also couples planning and scheduling more tightly; rule-based scheduling is therefore used as a controlled abstraction.
- **End-to-end evaluation vs. component-level intervention:** end-to-end scores alone cannot reliably attribute failure to specific components.[11]

Overall, the design explicitly trades some ecological realism for causal interpretability and reproducible comparison.[10]

Assumptions and limitations. The methodology assumes that (i) memory, retrieval, and scheduling explain a meaningful share of behavioural variance, (ii) imposed controls are suf-

ficient to approximate factor independence, and (iii) mock environments retain the structural properties needed for valid tool-interaction analysis. These assumptions constrain external validity; conclusions are therefore limited to relative component effects under controlled conditions rather than broad claims about all real-world agent deployments.

To operationalise this design, the execution workflow proceeds in eight stages:

1. Build a modular, containerised evaluation environment with mock MCP servers for database, file, and API tasks.
2. Define fixed task protocols and lock the shared seed set, perturbation schedules, and runtime settings.
3. Instantiate agent variants by changing only the three target components.
4. Run the full 2^3 factorial experiment with repeated matched trials.
5. Log complete trajectories, including tool calls, intermediate states, errors, and recovery actions.
6. Compute outcome and trajectory-level metrics.
7. Audit a sampled trace subset with the predefined diagnostic rubric, masked configuration identity where feasible, and second-pass consistency checks.
8. Estimate main/interaction effects with response-appropriate blocked or mixed-effects models, and report pooled and task-family-specific effect sizes with confidence intervals.

3.2. Expected Outcomes

The project is expected to produce both tangible research artefacts and scientific contributions. The anticipated outcomes are as follows:

- **A modular, reproducible evaluation framework.** The project will deliver a containerised benchmarking environment with mock MCP servers that mitigates the realism–reproducibility trade-off and enables controlled comparison of agent architectures beyond conventional end-to-end benchmarks.
- **Systematic quantification of component-level effects (RQ1).** Using the full 2^3 factorial design with repeated matched trials, the study will estimate pooled and task-family-specific main and interaction effects showing how memory, retrieval, and scheduling components influence task performance and failure behaviour under controlled conditions.
- **Diagnostic value of trajectory-level metrics (RQ2).** The study will evaluate whether trajectory signals (e.g., goal drift, repetition, and recovery behaviour) provide more informative and actionable component-level diagnosis than outcome-only metrics, using a structured manual audit of sampled traces with masked configuration identity where feasible, rubric-based scoring of failure-label alignment, specificity, and actionability, and comparative case analyses.
- **A reusable ablation testbed for future agent research.** The project will provide a reproducible evaluation setup—including mock environments, shared seeds, pre-generated perturbation schedules, and a standardised evaluation pipeline—to support follow-on controlled studies of agent architectures.

Collectively, these outcomes aim to strengthen methodological rigor in agent evaluation while improving the practical reliability of evidence used to guide architecture design decisions.

3.3. Evaluation Criteria

Evaluation will be based on predefined criteria aligned with the research questions and project deliverables:

- **Accuracy and analytical validity.** The framework should detect meaningful performance differences between component configurations and identify main/interaction effects with statistically defensible estimates across repeated matched trials, using analyses that preserve task–seed–schedule blocking and report both pooled and task-family-specific results.
- **Efficiency.** Each configuration will be assessed for execution latency and computational cost (including token usage), with comparisons reported relative to baseline configurations.
- **Diagnostic quality.** Trajectory diagnostics should provide more informative and actionable component-level diagnosis than outcome-only metrics; this is assessed with a predefined rubric covering failure-label alignment, specificity, and actionability, masked configuration identity where feasible, and a held-out subset rescored in a second masked pass for consistency.
- **Compliance with project requirements.** Experiments must satisfy the controlled-evaluation protocol: fixed model backbone, fixed prompts/tasks/interfaces, deterministic mock MCP environment conditional on shared seeds and perturbation schedules, complete trace logging, and reproducible analysis outputs.

Success is achieved when the framework is not only statistically rigorous and computationally transparent, but also practically useful for guiding agent architecture decisions under controlled conditions.

3.4. Risk Assessment and Mitigations

Potential technical, organisational, and resource risks are identified below, together with mitigation strategies designed to minimise impact:

- **Risk: Environmental instability ("bit rot").** Reliance on live APIs or changing web environments can introduce unpredictable variation and reduce consistency of evaluation results.
  Mitigation: The framework uses a fully containerised setup with mock MCP servers to ensure reproducible task execution and deterministic behaviour conditional on fixed seeds and perturbation schedules.
- **Risk: Computational and budget constraints.** Running multiple component configurations across tasks and seeds may exceed available compute resources or API budget limits.
  Mitigation: The study retains the full 2^3 design, but reduces task volume or repeated trials if resource pressure increases so that factor estimability is preserved.
- **Risk: Evaluation variance and non-determinism.** High variability in LLM outputs may obscure the true effect of individual components and weaken interpretation.
  Mitigation: Experiments are repeated over a shared fixed seed set with matched perturbation schedules, and results are analysed with response-appropriate blocked or mixed-effects models, confidence intervals, and trajectory-level metrics for robustness.
- **Risk: System integration complexity.** Building a modular architecture with interchangeable components and consistent logging may introduce integration delays or implementation errors.
  Mitigation: Development is staged: a minimal working pipeline is validated first, then components are integrated incrementally to reduce the risk of late-stage system failure.

This risk plan will be reviewed at each milestone so that mitigation actions can be adjusted early if execution conditions change.

3.5. Responsible Research Including Ethics

The project adopts a "safety-by-design" research process to ensure that evaluation gains do not come at the cost of privacy, fairness, or responsible deployment practice. Because the work

focuses on controlled benchmarking rather than real-user decision making, ethical risk is lower than in production systems, but non-trivial risks remain in data handling, bias propagation, and potential misuse of results.

Responsible research controls are built into the protocol as follows:

- **Data privacy and governance.** Experiments use synthetic or public benchmark data wherever possible, and avoid storing unnecessary personal data. Any logs containing user-like inputs are minimised, pseudonymised where needed, and retained only for reproducibility and audit purposes under institutional policy.
- **Human input and consent.** The core proposal does not require external human participants: diagnostic quality is assessed through an author-led rubric-based manual audit of a sampled trace subset, with masked configuration identity where feasible and a second masked pass on a held-out subset for consistency checks. If later extensions add external raters, participation will be voluntary, informed consent will be collected, and ethics guidance will be followed before recruitment.
- **Safety and misuse prevention.** The evaluation environment uses sandboxed/mock MCP tools and excludes high-risk autonomous actions (e.g., uncontrolled external transactions), reducing the chance that benchmarking pipelines can be repurposed for harmful automation.
- **Fairness and transparency.** Results are reported with uncertainty estimates, failure analyses, and explicit limitations (including benchmark bias and external-validity constraints) to avoid overstating general capability claims.
- **Environmental sustainability.** Compute usage is monitored via runtime/token accounting, and experiments are budgeted by capping task volume and repeated trials to limit unnecessary energy consumption without sacrificing the full factorial structure.

The project will follow university ethics guidance and complete any required ethics review or self-assessment before any protocol change that introduces identifiable personal data, non-anonymous human participants, or external expert raters.

4. Research Plan, Milestones, and Deliverables

4.1. Project Plan and Timeline (12 Weeks)

The project is organised into five phases over a 12-week schedule. The phase summaries below describe the high-level plan, while Figure 1 decomposes the work into dependent subtasks and Tables 1 and 2 summarise milestone and deliverable deadlines.

- **Phase 1: Infrastructure & Baseline Setup (Weeks 1–4). Tasks:** specify the evaluation protocol and tool schemas, implement mock MCP servers, build the logging and trace pipeline, and integrate a stable baseline agent. **Milestone (M_1):** a reproducible environment capable of fixed-seed / fixed-schedule baseline execution and reliable trace collection.
- **Phase 2: Modular Component Integration (Weeks 5–6). Tasks:** implement interchangeable memory backends, retrieval variants, and scheduling policies, then validate controlled component substitution end to end. **Milestone (M_2):** a modular agent architecture that supports clean component swaps without interface changes.
- **Phase 3: Experimental Execution & Iteration (Weeks 7–9). Tasks:** run pilot trials, calibrate seeds and runtime settings, execute the factorial experiment, and rerun failed configurations while checking log completeness and dataset quality. **Milestone (M_3):** a verified experimental dataset of execution traces and performance metrics.
- **Phase 4: Analysis & Diagnostics (Weeks 10–11). Tasks:** compute trajectory-level metrics, estimate main and interaction effects using response-appropriate blocked or mixed-effects models, and conduct failure analysis to interpret recovery behaviour and component interactions. **Milestone (M_4):** an analysis package with interpretable component effects and validated trajectory diagnostics.

- **Phase 5: Write-up & Finalisation (Week 12). Tasks:** consolidate results into the dissertation, refine the argumentation and limitations discussion, and prepare the final submission package. **Milestone (M_5):** final dissertation submission.

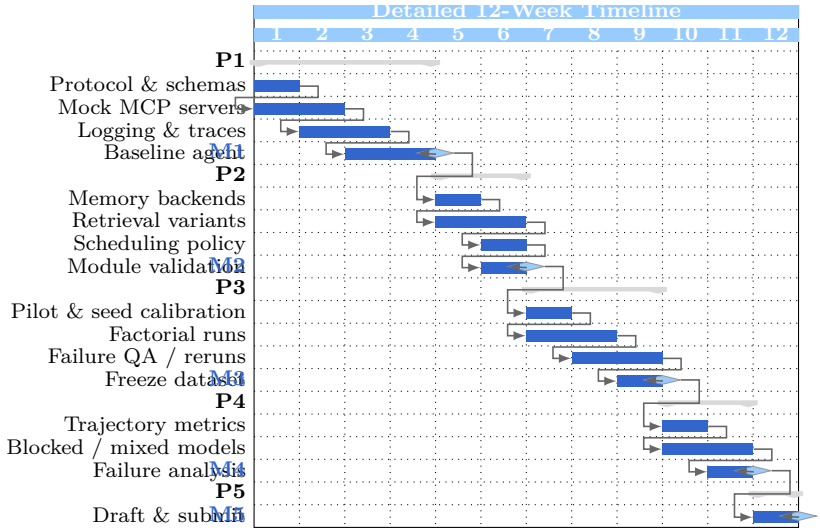

Figure 1: Detailed 12-week Gantt chart with decomposed tasks and milestone dependencies across P1–P5 (infrastructure, modular integration, experimental execution, analysis, and writing).

Milestone	Week	Description
M_1	4	Reproducible benchmark environment with validated logging, schemas, and fixed-seed / fixed-schedule baseline execution.
M_2	6	Modular architecture with interchangeable memory, retrieval, and scheduling components.
M_3	9	Complete experimental dataset with rerun tracking and quality-checked execution logs.
M_4	11	Statistical and diagnostic analysis completed with interpreted component effects.
M_5	12	Dissertation finalised and submitted.

Table 1: Project milestones for the revised 12-week plan.

Deliverable	Week	Description
D_1	4	Containerised benchmark environment, tool schemas, trace pipeline, and baseline agent run scripts.
D_2	6	Validated modular implementation of memory, retrieval, and scheduling variants.
D_3	9	Curated execution-trace dataset with pilot logs, rerun records, and quality checks.
D_4	11	Analysis report covering trajectory diagnostics, effect sizes, and interaction estimates.
D_5	12	Final dissertation document and reproducibility artefacts.

Table 2: Project deliverables aligned with phase completion.

References

[1] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. Evaluation and benchmarking of LLM agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*, pages 6129–6139, New York, NY, USA, 2025. Association for Computing Machinery.

[2] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023. Published in ICLR 2024.

[3] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024. Accepted by ICLR 2025 (formerly OpenDevin).

[4] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The BrowserGym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467*, 2024.

[5] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.

[6] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

[7] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. StableToolBench: Towards stable large-scale benchmarking on tool learning of large language models. *arXiv preprint arXiv:2403.07714*, 2024.

[8] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024.

[9] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. MCP-Universe: Benchmarking large language models with real-world model context protocol (MCP) servers. *arXiv preprint arXiv:2508.14704*, 2025.

[10] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025.

[11] Luca Gioacchini, Giuseppe Siracusano, Davide Sanvito, Kiril Gashteovski, David Friede, Roberto Bifulco, and Carolin Lawrence. AgentQuest: A modular benchmark framework to measure progress and improve LLM agents. *arXiv preprint arXiv:2404.06411*, 2024. Accepted at NAACL-HLT 2024.

[12] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, and Eugene Siow. MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers. *arXiv preprint arXiv:2508.20453*, 2025.

[13] Atsuyuki Miyai, Zaiying Zhao, Kazuki Egashira, Atsuki Sato, Tatsumi Sunada, Shota Onohara, Hiromasa Yamanishi, Mashiro Toyooka, Kunato Nishina, Ryoma Maeda, Kiyoharu Aizawa, and Toshihiko Yamasaki. WebChoreArena: Evaluating web browsing agents on realistic tedious web tasks. *arXiv preprint arXiv:2506.01952*, 2025.

[14] Yixing Jiang, Kameron C. Black, Gloria Geng, Danny Park, James Zou, Andrew Y. Ng, and Jonathan H. Chen. MedAgentBench: A realistic virtual EHR environment to benchmark medical LLM agents. *arXiv preprint arXiv:2501.14654*, 2025.

[15] Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, and Segev Shlomov. ST-WebAgentBench: A benchmark for evaluating safety and trustworthiness in web agents. *arXiv preprint arXiv:2410.06703*, 2024.

[16] Jiayi Pan, Yichi Zhang, Nicholas Tomlin, Yifei Zhou, Sergey Levine, and Alane Suhr. Autonomous evaluation and refinement of digital agents. *arXiv preprint arXiv:2404.06474*, 2024. Published at COLM 2024.

This section does not count towards the page limit. References should include embedded DOI links where available.